Module 4：pandas 與資料清理 — 從原始表格到可分析資料

所屬課程： Python 數據分析與 AI 工程基礎
模組編號： M4
時長： 3 小時
難度： 中階入門
前置模組： M3 NumPy 與陣列思維
後續模組： M5 視覺化與 EDA

一、模組定位

NumPy 給了你計算引擎，pandas 給了你工作台。

真實世界的資料不是整齊的數值矩陣：它有欄位名稱、有混合型別、有缺失值、有格式不統一的日期欄、有意義
pandas 完成這段路程。

這個模組教你四件事：

1. 認識 DataFrame：結構、索引、型別推斷的陷阱
2. 分析操作四步驟：篩選、排序、聚合、轉換
3. 資料整合：merge/join、concat、pivot table、時間序列基礎
4. 資料清理：缺失值、離群值、型別修正，把「髒資料」變成「可信資料」

二、學習目標

完成本模組後，學生能夠：

1. 說明 DataFrame 與 Series 的結構，以及 index 在 pandas 中扮演的角色

2. 執行完整的資料初始偵察流程（5 個必備函式）
3. 熟練使用 boolean indexing、sort_values、groupby + agg 處理業務分析問題
4. 使用 merge/join 合併多個 DataFrame，理解不同 join 類型的語意差異
5. 建立 pivot table 完成交叉統計分析
6. 處理時間序列資料的基本操作（解析日期、計算時間差、重採樣）
7. 識別並處理缺失值與離群值，使資料具備可分析狀態
8. 說明 pandas 2.0 的 Copy-on-Write 機制與 Arrow backend 的意義

三、關鍵概念清單

學完本模組應能用自己的話解釋以下所有詞彙：

- [ ] DataFrame vs Series（結構與關係）
- [ ] index（索引不是欄位，是列的身份證）
- [ ] dtype inference（型別推斷陷阱：為什麼 CSV 讀進來型別可能不對）
- [ ] boolean indexing（布林篩選，`&` 而非 `and`）
- [ ] .groupby() 的拆-套-合邏輯（split-apply-combine）
- [ ] merge vs concat（合併的兩種方式）
- [ ] left / right / inner / outer join（四種 join 的語意差異）
- [ ] pivot_table（交叉統計）
- [ ] datetime64 與 pd.to_datetime（時間序列的第一步）
- [ ] 缺失值的三種來源與四種處理策略
- [ ] IQR 離群值識別法
- [ ] Copy-on-Write（pandas 2.0 的鏈式賦值問題）
- [ ] Arrow backend（為什麼記憶體減半）
- [ ] 工具規模感：pandas / Polars / DuckDB / Spark 的邊界

四、投影片大綱（12 張）

編號	標題	核心主張
S1	DataFrame：分析世界的工作台	表格不是 Excel，是帶型別的計算物件
S2	讀進來、看一眼、查型別：每個專案開始之前的第一件事	都從這裡開始，先摸清楚手上的材料
S3	篩選與排序：精確地取出你要的資料	boolean indexing 是 pandas 的核心操作
S4	聚合與轉換：從資料到洞察的橋樑	groupby 的本質是業務切片思維
S5	merge 與 concat：把多張表合成一張	合併資料是現實工作中最常見的操作之一
S6	pivot_table：從長格式到交叉分析	換一個視角看資料，問題就清晰了
S7	時間序列基礎：讓日期欄真的變成日期	時間是資料中最特殊的維度
S8	缺失值：識別、理解、處理	沒處理的缺失值是分析毒藥
S9	離群值：識別、判斷、決策	不是所有極端值都是錯誤，但你必須知道它們在
S10	型別修正與資料格式統一	清理的本質是讓資料「說的語言」統一
S11	里程碑：pandas 2.0 — Copy-on-Write 與 Arrow backend	架構層級的升級解決了長達十年的語意問題
S12	工具規模感：小資料到大資料的工具選擇	知道什麼時候換工具，和知道怎麼用工具一樣重

五、逐張投影片詳述

S1 — DataFrame：分析世界的工作台

核心主張： DataFrame 不是試算表軟體的替代品，而是一個帶有型別系統、索引機制與豐富操作 API 的計算物件。

講師講解要點：

- DataFrame 的核心結構：列（rows）與欄（columns）。每個欄是一個 Series（一維帶標籤陣列），每個 Series 有獨立的 dtype。這與 ndarray 不同：ndarray 整個只能有一個 dtype，DataFrame 每欄可以不同。
- Index 的概念：index 不是一個欄位，它是列的「身份證」。預設是 0, 1, 2... 但可以是日期、字串、多層級索引。很多初學者因為不理解 index 而在 merge/join 時遇到意想不到的結果。
- DataFrame 是「帶標籤的矩陣」：分析時用欄名操作，計算時底層是 ndarray（或 Arrow array）。
- 建立 DataFrame 的三種常見方式：

```
import pandas as pd
import numpy as np

# 從字典建立
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'salary': [50000.0, 75000.0, 60000.0],
    'is_manager': [False, True, False]
})

# 從 CSV 檔案
df = pd.read_csv('data.csv')

# 從 NumPy 陣列
arr = np.random.randn(4, 3)
df = pd.DataFrame(arr, columns=['A', 'B', 'C'])

# DataFrame 的形狀
print(df.shape)  # (列數, 欄數)
```

```
print(df.columns)        # [illegible]
print(df.index)          # [illegible]
print(df.dtypes)         # [illegible]
```

視覺建議：一個 DataFrame 的解剖圖：左側標出 index 欄（帶標籤）、頂部標出 columns（欄名）、每欄下方標出 dtype、每個 Series 用不同顏色框起。旁邊附與 Excel 試算表的對比說明，強調「DataFrame 的欄有型別，Excel 的格沒有」。

轉場：「有了 DataFrame，第一件事是什麼？讀進資料，然後看一眼手上的材料。」

S2 — 讀進來、看一眼、查型別：每個專案都從這裡開始

核心主張：
資料分析的第一步不是計算，是偵察。在對資料一無所知時，有一套標準的「初次見面流程」。

講師講解要點：

- 五個必備的初始偵察函式：

```
df.head()        # [illegible] 5 [illegible]
df.info()        # [illegible]
df.describe()    # [illegible]count/mean/std/min/max[illegible]
df.shape         # ([illegible], [illegible])
df.dtypes        # [illegible]
```

- **`df.info()`**
  是最重要的那一個：它同時告訴你欄名、型別、非空數量（從而反推缺失數量）、記憶體使用量。一眼看完
  object）。
- 型別推斷陷阱：pandas 讀 CSV 時會猜型別。常見問題：
- 含有「NT$1,200」的金額欄被讀成 object 而非 float
- 含有「2023-01-15」的日期欄被讀成 object 而非 datetime64
- 含有整數的欄位因為有一個缺失值而變成 float64（因為 NaN 是 float）
- 含有 "True"/"False" 字串的布林欄被讀成 object
- `df.value_counts()` 用於類別型欄位的分布快覽，是 `describe()` 對非數值欄位的補充。

- 培養「資料第一眼的直覺」：看完 `info()`

後，腦子裡要有一張評估表：缺失值多嗎？有沒有型別異常？欄位名稱有沒有空格或特殊字元？樣本量夠嗎

視覺建議：一份模擬的 `df.info()` 輸出截圖，用不同顏色標注關鍵資訊：欄名（藍）、dtype（橙）、非空計數（綠，並標示哪些有缺失）、記憶體 checklist：五個函式依序列出，每個旁邊標注「告訴你什麼」。

轉場：「確認資料長什麼樣子之後，接下來是分析師的日常：篩選和排序。」

S3 — 篩選與排序：精確地取出你要的資料

核心主張：boolean indexing 是 pandas 的核心操作語言。掌握篩選與排序，就能從任意角度切入資料。

講師講解要點：

- 篩選（Boolean Indexing）：

```
# 單條件
young = df[df['age'] < 30]

# 多條件用 & 和 | 不是 and 和 or
# 一定要加括號
result = df[(df['age'] > 25) & (df['salary'] > 60000)]

# 或用 query 語法
result = df.query("age > 25 and salary > 60000")

# 成員
result = df[df['department'].isin(['Engineering', 'Design'])]

# 字串
result = df[df['name'].str.contains('Alice')]
result = df[df['city'].str.startswith('Taipei')]
```

- 排序（Sorting）：

```
# 單欄
df.sort_values('salary', ascending=False)

# 多欄
```

```
# [redacted]
df.sort_values(['department', 'salary'], ascending=[True, False])
[redacted]
# [redacted] index [redacted]
df.sort_index()
```

- `sort_values` 預設回傳新的 DataFrame（不改動原始資料）。這是 pandas 的一般設計原則：大部分操作回傳新物件。
- `loc` vs `iloc`：`loc` 用標籤索引（欄名、index 值），`iloc` 用整數位置索引。

```
df.loc[0, 'salary']          # [redacted] 0 [redacted] salary [redacted]
df.iloc[0, 2]                # [redacted] 0 [redacted] 2 [redacted]
df.loc[0:5, ['name', 'salary']]  # [redacted] 6 [redacted]
```

視覺建議：一個 DataFrame 示意圖，用不同顏色高亮標示篩選結果的列（符合條件的）、排序後的順序箭頭、loc/iloc 取值的目標格子。旁邊附常見的錯誤：`df[df['age'] > 25 and df['salary'] > 60000]` 會報錯，解釋原因並給出正確寫法。

轉場：「篩選之後，我們看更強大的聚合工具：groupby。」

S4 — 聚合與轉換：從資料到洞察的橋樑

核心主張：groupby 的本質是「split-apply-combine」三步驟，它把資料問題轉換成業務問題：按什麼維度切、對每個切片做什麼、

講師講解要點：

- groupby 三步驟：Split（按欄位分組）→ Apply（對每組計算）→ Combine（合併結果）

```
# [redacted] groupby
df.groupby('department')['salary'].mean()
[redacted]
# [redacted]
df.groupby(['region', 'category'])['revenue'].sum()
[redacted]
# agg[redacted]
df.groupby('department').agg({
```

```
    'salary': ['mean', 'std', 'count'],
    'performance': 'median'
})

# agg 支援多種聚合方式
df.groupby('department').agg(
    avg_salary=('salary', 'mean'),
    num_employees=('salary', 'count'),
    med_performance=('performance', 'median')
)
```

- transform vs agg 的差異：

```
# agg 會縮減資料
dept_avg = df.groupby('department')['salary'].mean()  # 只剩部門數

# transform 保留原 df 的長度
# 適合用來計算相對值
df['salary_ratio'] = df['salary'] / df.groupby('department')['salary'].transform('mean')
```

- 常用聚合函式：`sum`、`mean`、`median`、`std`、`count`、`nunique`（唯一值數量）、`first`、`last`、`min`、`max`
- 業務問題直接翻譯：「各城市的訂單數量和平均金額」→ `df.groupby('city').agg({'order_id': 'count', 'amount': 'mean'})`

視覺建議：
一張三步驟示意圖：左邊是一個混合的銷售資料表，中間被虛線切成顏色不同的子表（Split），右邊子表各自計 + Combine）。整個圖標記 split-apply-combine 三個步驟。

轉場：「單張表的操作學完了。現實中的資料幾乎都要從多張表合併而來。」

S5 — merge 與 concat：把多張表合成一張

核心主張：資料整合是現實工作中最常見的任務之一。merge 處理有關聯的表，concat 處理結構相同的表。

講師講解要點：

- concat：垂直或水平堆疊

```
# [redacted]
df_q1 = pd.read_csv('sales_q1.csv')
df_q2 = pd.read_csv('sales_q2.csv')
df_all = pd.concat([df_q1, df_q2], ignore_index=True)

# [redacted]
df_combined = pd.concat([df_features, df_labels], axis=1)
```

- merge：根據欄位關聯兩張表

```
# orders [redacted] customer_id[redacted]customers [redacted]
orders = pd.DataFrame({'order_id': [1,2,3], 'customer_id': [101, 102, 101], 'amount': [500, 300,
customers = pd.DataFrame({'customer_id': [101, 102, 103], 'name': ['Alice', 'Bob', 'Charlie'], 'c

# inner join[redacted]
result = pd.merge(orders, customers, on='customer_id', how='inner')

# left join[redacted] NaN[redacted]
result = pd.merge(orders, customers, on='customer_id', how='left')

# [redacted]
result = pd.merge(df1, df2, left_on='user_id', right_on='customer_id')
```

- 四種 join 類型的語意差異：

join 類型	保留哪些列	典型使用場景
`inner`	兩邊都有的	只分析有完整資料的記錄
`left`	左表所有列	保留主表，補充右表資訊
`right`	右表所有列	較少用，通常改用 left
`outer`	兩邊所有列	檢查哪些記錄只存在於一邊

- 常見錯誤：重複欄位名稱導致 merge 後出現 `_x` 和 `_y` 後綴，需要在 merge 前重命名欄位或用 `suffixes` 參數指定。

視覺建議：四個並排的 Venn 圖，分別標示 inner/left/right/outer join 保留的範圍（陰影部分），每個圖下方附上對應的程式碼片段。

轉場：「合併完資料，有時你需要從不同角度看它——這就是 pivot table 的用途。」

S6 — pivot_table：從長格式到交叉分析

核心主張：pivot table 把「長格式」的流水帳資料，轉換為二維的交叉統計表，讓業務問題一目了然。

講師講解要點：

- 長格式 vs 寬格式：原始資料通常是長格式（每列是一筆事件記錄），pivot table 把它轉成寬格式（行是一個維度，欄是另一個維度）。

```
# 長格式
sales = pd.DataFrame(
    {'region': ['North', 'North', 'South', 'South', 'East'],
    'product': ['A', 'B', 'A', 'B', 'A'],
    'quarter': ['Q1', 'Q2', 'Q1', 'Q1', 'Q2'],
    'revenue': [100, 200, 150, 300, 250]
})

# pivot table
pivot = sales.pivot_table(
    values='revenue',
    index='region',
    columns='product',
    aggfunc='sum',
    fill_value=0,        # 缺值補 0
)

# 結果：region 列，product 欄，revenue 值

# 多維度
pivot_multi = sales.pivot_table(
    values='revenue',
    index='region',
    columns='quarter',
    aggfunc=['sum', 'mean']
)

# 回到長格式
long = pivot.reset_index().melt(id_vars='region', var_name='product', value_name='revenue')
```

- `margins=True` 參數加入總計列和欄，等同於 Excel 的「加總」功能。

- pivot_table 與 groupby 的選擇：如果你想要一個二維的交叉表格，用 pivot_table；如果你只需要一維的聚合，用 groupby。

視覺建議： 一張從長格式到寬格式的轉換示意圖：左邊是流水帳格式的原始資料表，右邊是 pivot table 的交叉結果，用箭頭連接，標示 `index`、`columns`、`values` 三個參數對應到圖中的哪個維度。

轉場：「下一個特殊維度是時間。日期欄的處理方式和一般欄位完全不同。」

S7 — 時間序列基礎：讓日期欄真的變成日期

核心主張： 時間是資料中最特殊的維度。把日期欄從字串變成 datetime64，打開了排序、計算時間差、重採樣等一系列分析能力。

講師講解要點：

- 解析日期：CSV 讀進來的日期幾乎一定是 object（字串），第一步是轉換：

```
# [redacted]
df = pd.read_csv('orders.csv', parse_dates=['order_date', 'ship_date'])

# [redacted]
df['order_date'] = pd.to_datetime(df['order_date'])
df['order_date'] = pd.to_datetime(df['order_date'], format='%Y/%m/%d')  # [redacted]
df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')  # [redacted] NaT
```

- 時間屬性萃取：

```
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.month
df['day_of_week'] = df['order_date'].dt.dayofweek  # 0=Monday
df['quarter'] = df['order_date'].dt.quarter
df['is_weekend'] = df['order_date'].dt.dayofweek >= [redacted]
```

- 計算時間差：

```
# [redacted]
df['shipping_days'] = (df['ship_date'] - df['order_date']).dt.days
```

```
# 計算年資
today = pd.Timestamp.today()
df['tenure_years'] = (today - df['hire_date']).dt.days / 365
```

- 時間序列重採樣（Resample）：

```
# 設定 index
df = df.set_index('order_date')

# 月營收
monthly = df['revenue'].resample('ME').sum()

# 季平均
quarterly = df['revenue'].resample('QE').mean()

# 7 日移動平均
df['revenue_7d_avg'] = df['revenue'].rolling(window=7).mean()
```

- 常見陷阱：時區問題（timezone-naive vs timezone-aware）；`NaT`（Not a Time）是時間版的 NaN，`pd.isnull()` 可以檢測。

視覺建議：一條時間軸示意圖，標示 resample 的不同頻率（每日、每週、每月）對應到的聚合結果。旁邊附一個 dt accessor 的常用屬性速查表（year/month/day/dayofweek/quarter/hour 等）。

轉場：「時間處理之後，我們進入清理工作的核心：缺失值。」

S8 — 缺失值：識別、理解、處理

核心主張：
沒有清理過的缺失值是分析毒藥。不同來源的缺失，需要不同的處理策略。亂填、亂刪都是錯的。

講師講解要點：

- 缺失值的三種來源：
- 資料收集時就沒有（例如問卷某題空白、選填欄位）
- 資料合併時的欄位不對齊（LEFT JOIN 後的 NaN）

系統故障或資料管道問題（sensor 未回傳值）

統計學的缺失值分類（MCAR / MAR / MNAR）：

- MCAR（Missing Completely At Random）：缺失和任何變數都無關。例如問卷被風吹走。可以直接刪除，不會產生偏差。
- MAR（Missing At Random）：缺失和「其他已知變數」有關，但和缺失值本身無關。例如年輕人較不填收入欄位。可以用其
- MNAR（Missing Not At Random）：缺失和「缺失值本身」有關。例如高收入者刻意不填收入。最棘手，填補可能產生偏差。

初學者實用判斷：大部分情況你不需要精確分類，但要問自己：「這些缺失是隨機的，還是有系統性原因？

識別缺失值：

```
# [redacted]
df.isnull().sum()
# [redacted]
df.isnull().mean()
# [redacted]
df[df.isnull().any(axis=1)]
# [redacted] seaborn [redacted] missingno [redacted]
```

- 四種處理策略：

```
# 1. [redacted]
df_clean = df.dropna()                    # [redacted]
df_clean = df.dropna(subset=['salary'])   # [redacted]
df_clean = df.dropna(thresh=5)            # [redacted] 5 [redacted]
# 2. [redacted]
df['city'] = df['city'].fillna('Unknown')
df['is_online'] = df['is_online'].fillna(False)
# 3. [redacted]
df['age'] = df['age'].fillna(df['age'].median())
# 4. [redacted]
df['price'] = df['price'].ffill()  # [redacted]
df['price'] = df['price'].bfill()  # [redacted]
```

- 選擇策略的原則：
- 缺失比例 < 5%：刪除通常可接受
- 缺失比例 5-30%：根據來源選擇填補策略
- 缺失比例 > 30%：評估這個欄位是否值得保留
- 缺失不是隨機的（例如高收入者更少填寫年齡）：統計填補可能引入偏差

視覺建議：一張資料框示意圖，標示有缺失的欄位（用灰色格子表示 NaN）。旁邊附一個決策樹：缺失值 → 缺失比例高低？→ 缺失是否隨機？→ 導向不同的處理策略。

轉場：「缺失值之後，我們看另一類問題：數值上的極端值。」

S9 — 離群值：識別、判斷、決策

核心主張：
不是所有極端值都是錯誤，但你必須知道它們在哪裡。離群值的處理需要結合業務理解，不能盲目刪除。

講師講解要點：

- IQR 方法（四分位距法）：

```
Q1 = df['salary'].quantile(0.25)
Q3 = df['salary'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# [illegible]
df['is_salary_outlier'] = (df['salary'] < lower_bound) | (df['salary'] > upper_bound)
outliers = df[df['is_salary_outlier']]
print(f"[illegible] {len(outliers)} [illegible]")
```

- Z-score 方法：

```
from scipy import stats  # [illegible] NumPy [illegible]

```

```
# scores = np.abs((df['salary'] - df['salary'].mean()) / df['salary'].std())
outliers = df[z_scores > 3]  # ■■■ 3 ■■■■
```

- 視覺化識別（配合 M5 視覺化）：箱形圖（boxplot）是最直觀的離群值檢視工具。
- 處理策略：

```
# 1. ■■■■■■■■■■■■
df = df[~df['is_salary_outlier']]

# 2. ■■■Winsorizing■■■■■■■■■■■■■■■■
df['salary_clipped'] = df['salary'].clip(lower=lower_bound, upper=upper_bound)

# 3. ■■■■■■■■■■■■■■■■
```

```
# [redacted]
# True/False, 1/0, 'yes'/'no', 'Y'/'N' [redacted]
df['is_manager'] = df['is_manager'].map({True: True, False: False, 1: True, 0: False, 'Y': True,
]
# [redacted] Categorica[redacted]
df['department'] = df['department'].astype('category')
df['city'] = df['city'].astype('category')
]
# [redacted] NaN [redacted] float64[redacted]pandas 2.0 [redacted]
# pandas 1.0+ [redacted] nullable integer type
df['age'] = df['age'].astype('Int64')  # [redacted] I[redacted] NaN
```

- 欄位名稱清理（現實中的 CSV 欄名往往有空格和特殊字元）：

```
# [redacted]
df.columns = df.columns.str.strip()          # [redacted]
df.columns = df.columns.str.lower()          # [redacted]
df.columns = df.columns.str.replace(' ', '_') # [redacted]
```

- 重複列處理：

```
# [redacted]
print(df.duplicated().sum())
|
# [redacted]
df = df.drop_duplicates()
|
# [redacted]
df = df.drop_duplicates(subset=['order_id'], keep='last')
```

- 清理完成的驗收標準：每欄的 dtype 符合預期；`isnull().sum()` 的結果是已知且有意識的；沒有任何重複的 primary key；所有數值欄的範圍在合理區間內。

視覺建議：一張清理前後對照表：左邊是一個「髒」的 DataFrame（有 NaN、型別錯誤、欄名有空格、疑似離群的數字），右邊是清理後的版本，每個修改位置用顏色標注與說明。

轉場：「資料清理完，我們來看 pandas 這個工具本身的重要演進。」

S11 — 里程碑：pandas 2.0 — Copy-on-Write 與 Arrow backend

核心主張：pandas 2.0 是架構層級的升級，Copy-on-Write 解決了長達十年的語意不一致問題，Arrow backend 帶來大幅度的記憶體效率提升。

講師講解要點：

- Copy-on-Write（CoW）：pandas 1.x 最臭名昭著的問題之一是「鏈式賦值」的不確定行為：

```
# pandas 1.x 的問題
df[df['age'] > 30]['salary'] = 0
# 這會修改 df 嗎？
# 答案：pandas 版本決定
# 會產生 undefined behavior，通常是 SettingWithCopyWarning

# pandas 2.0 Copy-on-Write 的解法
# 明確的寫法
df.loc[df['age'] > 30, 'salary'] = 0

# 或者用 where 保留原資料
df['salary_modified'] = df['salary'].where(df['age'] <= 30, 0)
```

- Arrow backend：pandas 2.0 允許使用 Apache Arrow 作為底層儲存格式（之前預設是 NumPy）。Arrow 格式帶來：
- 支援 nullable integer（原生 NaN，而非用 float 來表示 int 的 NaN）
- 字串效率大幅提升
- 記憶體使用量在典型資料集上可減少 30-50%

```
# 啟用 Arrow backend
df = pd.read_csv('data.csv', dtype_backend='pyarrow')

# 檢查資料型態
print(df.dtypes)  # 顯示 ArrowDtype
```

- 對學習者的實際意義：
- 遇到 `SettingWithCopyWarning` 不要忽略，這是 pandas 在告訴你賦值可能無效
- 在 pandas 2.0 環境下，使用 `.loc` 做有條件的賦值是最安全的寫法
- 如果處理大型資料集，試試 `dtype_backend='pyarrow'` 看是否能減少記憶體用量

視覺建議：兩個並排的程式碼對比框。左框「pandas 1.x 的語意地雷」，展示鏈式賦值，下方顯示警告訊息。右框「pandas 2.0 正確寫法」，展示 `.loc` 賦值，下方顯示「語意清晰，結果可預期」。底部附 Arrow backend 的記憶體節省柱狀比較示意。

轉場：「pandas 很強，但有邊界。最後一張投影片，我們談工具規模感。」

S12 — 工具規模感：小資料到大資料的工具演進

核心主張：pandas 是小到中型資料的最佳選擇，但知道它的邊界以及邊界之後有什麼工具，是工程師成熟度的標誌。

講師講解要點：

- pandas 的適用範圍：記憶體內（in-memory）操作，資料集在幾百 MB 到幾 GB 之間（取決於機器記憶體）。速度對大多數分析任務足夠，生態最成熟，教學資源最豐富。在 Python 資料分析的語境下，pandas 是預設選擇，也是你要最熟練的工具。
- Polars：用 Rust 撰寫的 DataFrame 庫，API 設計上借鑒 pandas 但更嚴格（沒有 index、強制 lazy evaluation 選項）。在單機上比 pandas 快 5-20 倍，Arrow 原生支援，適合需要高效能但資料仍在單機的場景。2023-2024 年快速崛起。
- DuckDB：內嵌式 SQL 分析引擎，可以直接查詢 CSV、Parquet、pandas DataFrame，不需要資料庫伺服器。適合數 GB 到數十 GB 的資料，SQL 介面對有資料庫背景的人非常友善。
- Apache Spark（PySpark）：分散式計算框架，適合 TB 級以上的資料，需要集群環境。PySpark DataFrame API 設計上與 pandas 相似，降低了遷移學習成本。
- 選擇框架的起點直覺：

資料量	推薦工具	理由
< 1 GB	pandas	生態最成熟，語法最易學
1–50 GB（單機）	Polars 或 DuckDB	速度與記憶體效率顯著提升

> 50 GB 或多機	Spark (PySpark)	分散式計算是唯一選項

視覺建議：一張工具規模尺，水平軸是資料量（從 MB 到 PB），上方標示不同工具的適用範圍（用彩色橫條表示）。每個工具下方加 1-2 個關鍵特徵描述。最底部加一行：「本課程專注 pandas，但你應該知道邊界在哪裡。」

六、練習設計

練習 1：DataFrame 操作基礎（S1-S4 結束後，25 分鐘）

情境說明：

以下是一份模擬的電商訂單資料，包含欄位：`order_id`、`customer_id`、`region`、`product_category`、`revenue`、`o`

任務清單：

1. 用下方字典建立 DataFrame，並執行初次偵察五步驟（`head`、`info`、`describe`、`shape`、`dtypes`）

```
import pandas as pd
data = {
    'order_id': range(1, 21),
    'customer_id': [f'C{i:03d}' for i in [101,102,103,101,104,102,105,103,101,102,
                                          104,105,101,103,102,104,105,101,102,103]],
    'region': ['North','South','East','North','West','South','East','North','West','South',
               'North','East','South','West','North','East','South','West','North','South'],
    'product_category': ['Electronics','Clothing','Food','Electronics','Books',
                         'Clothing','Food','Electronics','Books','Clothing',
                         'Electronics','Food','Clothing','Books','Electronics',
                         'Food','Clothing','Books','Electronics','Clothing'],
    'revenue': [1200,350,80,900,45,420,60,1500,30,380,
                1100,70,290,25,800,90,340,40,1300,410],
    'order_date': pd.date_range('2024-01-01', periods=20, freq='3D'),
}
df = pd.DataFrame(data)
```

1. 篩選出 `region == 'North'` 且 `revenue > 500` 的訂單（使用 boolean indexing，用 `&`）
2. 篩選出 `product_category` 為 'Electronics' 或 'Clothing' 的訂單（使用 `.isin()`）

3. 計算每個 `product_category` 的訂單數量、總營收、平均訂單金額，用 `groupby + agg` 一次完成
4. 按總營收降序排列，找出營收最高的前三個產品類別
5. 建立一個新欄位 `revenue_tier`：使用 `pd.cut` 或 `np.select` 把 revenue 分為 'Low'（< 200）、'Medium'（200-800）、'High'（> 800）
6. 計算每個 `region` 的平均訂單金額，並找出最高和最低的地區

驗收標準： - 能正確使用 boolean indexing 的多條件組合（`&` 而非 `and`） - groupby 的 agg 結果包含三個指標（count、sum、mean） - 新欄位的型別合理（Categorical 或 object）

練習 2：資料合併與 pivot table（S5-S6 結束後，25 分鐘）

情境說明：
你有兩張表：訂單表和顧客資料表，需要合併後進行交叉分析。

任務清單：

```
orders = pd.DataFrame(
    'order_id': range(1, 11),
    'customer_id': [101,102,103,101,104,102,105,103,101,106],
    'product': ['A','B','A','C','B','A','C','B','A','B'],
    'amount': [500,300,700,450,250,600,800,350,550,400],
    'month': ['Jan','Jan','Feb','Feb','Mar','Mar','Jan','Feb','Mar','Jan'],
)

customers = pd.DataFrame(
    'customer_id': [101,102,103,104,105],
    'city': ['Taipei','Taipei','Kaohsiung','Taichung','Taipei'],
    'membership': ['Gold','Silver','Bronze','Gold','Silver'],
)
```

1. 用 `inner join` 合併兩張表（`on='customer_id'`），確認結果的 shape
2. 用 `left join` 合併，找出哪些 order 的 customer 不在 customers 表中（觀察 NaN 出現的位置）
3. 用 `pivot_table` 計算每個城市（index）、每個產品（columns）的總銷售金額（values），`fill_value=0`
4. 在 pivot_table 中加入 `margins=True`，觀察總計列和欄

5. 建立另一個 pivot_table：每個月份（index）、每個會員等級（columns）的平均訂單金額
6. 將第 3 步的 pivot table 用 `reset_index().melt(...)` 轉換回長格式，確認欄位名稱正確

驗收標準： - 理解 inner 和 left join 結果列數不同的原因 - pivot_table 的三個核心參數（index/columns/values）使用正確 - melt 轉換回長格式後的結果可以重新建立原始 pivot

---

練習 3：Mini 資料清理工作坊（S7-S10 結束後，45 分鐘）

情境說明：
你拿到一份「髒」的員工資料，目標是完成一份符合分析品質的清理作業，並回答三個業務問題。這是本模組的

資料說明：
欄位包含：`employee_id`、`department`、`hire_date`（有些被讀成字串）、`salary`（有些包含「NT$」前綴與逗號）分，但有幾個值是 99）、`city`、`is_manager`（混用 True/False 和 1/0）、`last_review_date`（有 15% 缺失）

```
import pandas as pd
import numpy as np

np.random.seed(42)
n = 40

data = {
    'employee_id': range(1001, 1001 + n),
    'department': np.random.choice(['Engineering','Marketing','Sales','HR','Finance'], n),
    'hire_date': pd.date_range('2015-01-01', periods=n, freq='45D').astype(str),
    'salary': [f'NT$({s:,})' for s in np.random.randint(40000, 120000, 30)] +
              [str(s) for s in np.random.randint(40000, 120000, 10)],
    'performance_score': [list(np.random.randint(1, 6, 35)) + [99, 99, 99, 99, 99]],
    'city': np.random.choice(['Taipei', 'Kaohsiung', 'Taichung', None], n, p=[0.5, 0.2, 0.2, 0.1]),
    'is_manager': [True, False, 1, 0, True] * 8,
    'last_review_date': list(pd.date_range('2023-01-01', periods=34, freq='10D').astype(str)) + [No
}
df_raw = pd.DataFrame(data)
```

任務清單（依序完成）：

Step 1：資料偵察 - 執行初次偵察五步驟，記錄：有幾個欄位有缺失、哪些欄位型別看起來不對

Step 2：型別修正 - 把 `hire_date` 轉為 datetime64 型別（使用 `pd.to_datetime`） - 把 `salary` 的 'NT$' 前綴和逗號去除，轉為 float（使用 `.str.replace` 和 `.astype`） - 把 `is_manager` 統一為 bool

型別（使用 `.map` 或 `.astype`）

Step 3：缺失值處理 - 確認 `city` 的缺失情況，用 'Unknown' 填補 - 確認 `last_review_date` 的缺失情況（應為約 15%），決定填補策略並說明理由

Step 4：離群值與錯誤值識別 - 用 IQR 方法識別 `salary` 的離群值，列出離群值的員工 ID - 把 `performance_score == 99` 識別為資料錯誤，替換為 `NaN`，再用中位數填補

Step 5：回答業務問題 1. 各部門的平均薪資是多少？用表格呈現，並指出最高與最低薪的部門 2. 管理職（`is_manager == True`）和非管理職的績效分布有何差異（用 `groupby + describe` 比較） 3. 計算每位員工從入職到今日（`pd.Timestamp.today()`）的年資，找出哪個部門的平均年資最長

驗收標準： - 清理後的 DataFrame：每欄的 dtype 正確、無非預期的缺失值、performance_score 無 99 - 缺失值處理有明確的策略說明（為什麼這樣處理） - 三個業務問題的答案都有對應的程式碼支撐 - 整個流程可以從頭到尾不報錯地執行

七、模組里程碑卡

```
[illegible] — pandas 2.0[illegible]2023 [illegible] 4 [illegible]

[illegible] pandas [illegible]
[illegible] df[mask][ 'col' ] = value [illegible]
Copy-on-Write [illegible]
[illegible]

[illegible]
[illegible] Copy-on-Write[illegible]
[illegible] Arrow backend[illegible] 30-50%
[illegible] nullable integer[illegible] float [illegible] int + NaN[illegible]
[illegible]

[illegible]
[illegible] SettingWithCopyWarning [illegible]
[illegible]
[illegible] pd.options.mode.copy_on_write = True [illegible]
```

八、本模組結語

pandas 是資料分析工作的核心工作台，也是你在資料科學旅程中用得最頻繁的工具之一。

你現在掌握的資料清理能力——識別型別錯誤、處理缺失值、找出離群值——是讓任何分析結果「可以被相信」的

接下來的 M5 視覺化與 EDA，會教你如何把清理後的資料變成圖表，讓模式和異常從數字堆裡浮現出來。

參考文獻與引用來源

pandas 官方資源

1. pandas 2.0 Release Notes (April 3, 2023) — Arrow backend、Copy-on-Write 語意變更。pandas.pydata.org/docs/whatsnew/v2.0.0.html
2. pandas Official Documentation — DataFrame API 完整說明。pandas.pydata.org/docs
3. Arrow backend 記憶體改善 — PyArrow 字串欄位相比 NumPy object 欄位可減少最多 70% 記憶體；整體使用 Arrow dtype_backend 約減少 50% 記憶體。來源：pandas 官方文件與社群基準測試。參見 datapythonista.me/blog/pandas-20-and-the-arrow-revolution-part-i

缺失值理論

1. MCAR / MAR / MNAR 分類 — 原始定義來自 Rubin, D.B. (1976). "Inference and Missing Data." Biometrika, 63(3), 581-592. 此三分類是缺失值處理的統計學標準框架。
2. Little, R.J.A. & Rubin, D.B. (2019). Statistical Analysis with Missing Data, 3rd Edition. Wiley. — 缺失值分析的權威教科書。

缺失值處理決策門檻

1. 缺失比例 < 5% / 5-30% / > 30% 門檻 — 此為資料科學實務中常用的經驗法則，非絕對標準。參考來源：Schafer, J.L. (1999). "Multiple imputation: a primer." Statistical Methods in Medical Research, 8(1), 3-15. 以及 Bennett, D.A. (2001). "How can I deal with missing data in my study?" Australian and New Zealand Journal of Public Health, 25(5), 464-469.

工具生態

1. Polars Documentation — 高效能 DataFrame 替代方案。docs.pola.rs
2. DuckDB Documentation — 嵌入式分析資料庫。duckdb.org/docs
3. Apache Arrow — 跨語言欄式記憶體格式。arrow.apache.org

延伸閱讀

1. McKinney, W. (2022). Python for Data Analysis, 3rd Edition. O'Reilly Media. — pandas 作者親撰的權威指南。
2. Wickham, H. (2014). "Tidy Data." Journal of Statistical Software, 59(10). — 整潔資料原則。doi.org/10.18637/jss.v059.i10